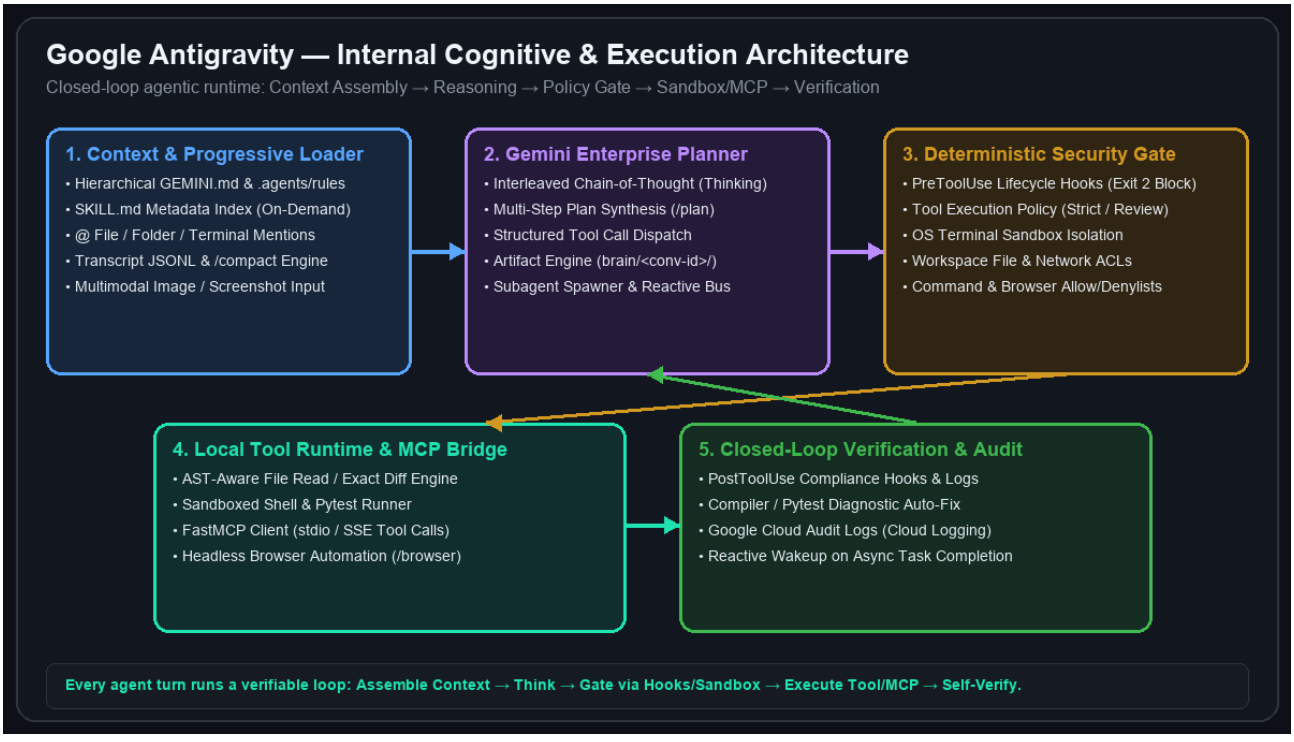
TRYOLABS • OFFICIAL INSTRUCTOR STUDY GUIDE & TECHNICAL REFERENCE

Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

Agentic Software Engineering with Google Antigravity

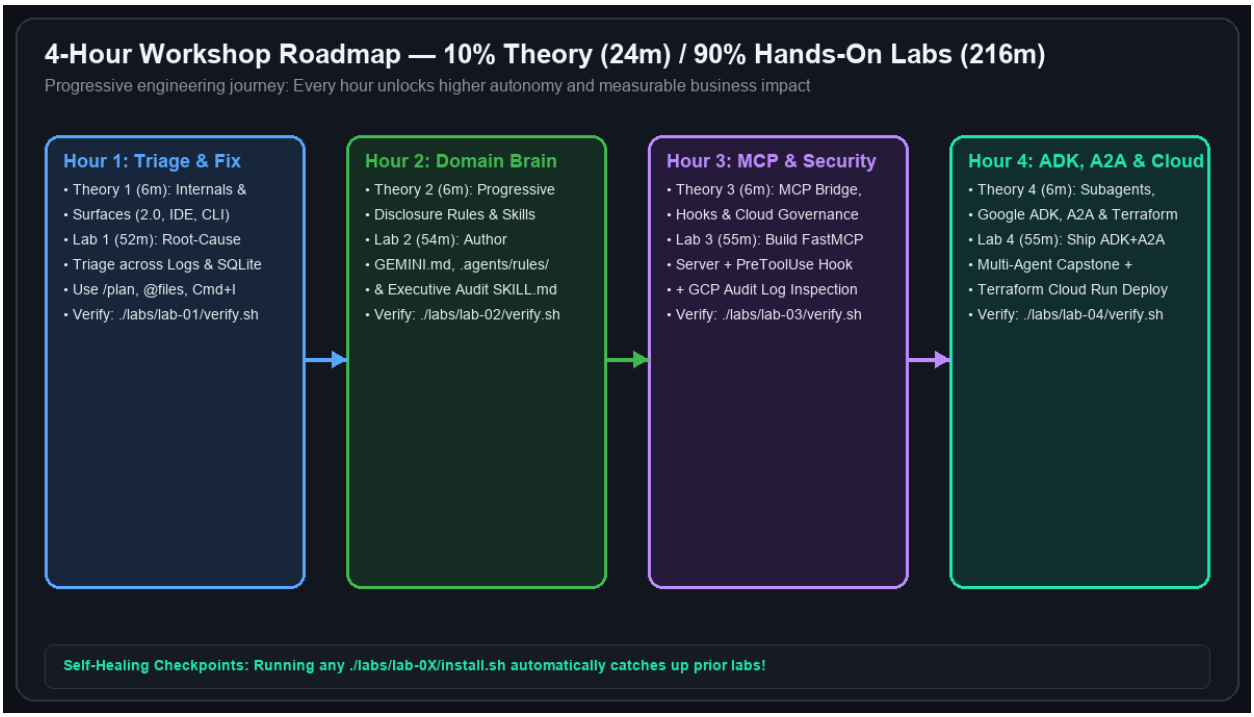
Complete 4-Hour Hands-On Developer Workshop Manual (10% Theory / 90% Labs) — Covering Antigravity Internals, Rules & Progressive Disclosure Skills, FastMCP & Lifecycle Hooks, Gemini Enterprise Cloud Governance, Multi-Agent Systems with Google ADK & A2A Protocol (`adk web` + Terraform), and the Master Acronym Glossary for the Telecom & Retail Editions.



How to Use This Study Guide: This document is written as a self-contained textbook for workshop instructors. Even if you have never worked in Telecommunications (5G RAN / BSS billing) or Omnichannel Retail (WMS / POS / Supply Chain), or if this is your first time teaching Google Antigravity, Google ADK, and the A2A Protocol, this guide explains every concept, architecture diagram, code bug, mathematical formula, security control, and domain acronym from first principles.

1. Workshop Structure, Pedagogy & 4-Hour Pacing (10% Theory / 90% Labs)

This workshop is strictly engineered for a **4-hour (240-minute)** delivery where **10% (24 minutes)** is dedicated to high-impact visual theory and **90% (216 minutes)** is spent in hands-on coding labs on attendees' local laptops. Instead of front-loading 24 minutes of slides at the beginning, the theory is split into **four 6-minute micro-capsules**, each immediately followed by a 52-to-55-minute progressive lab.



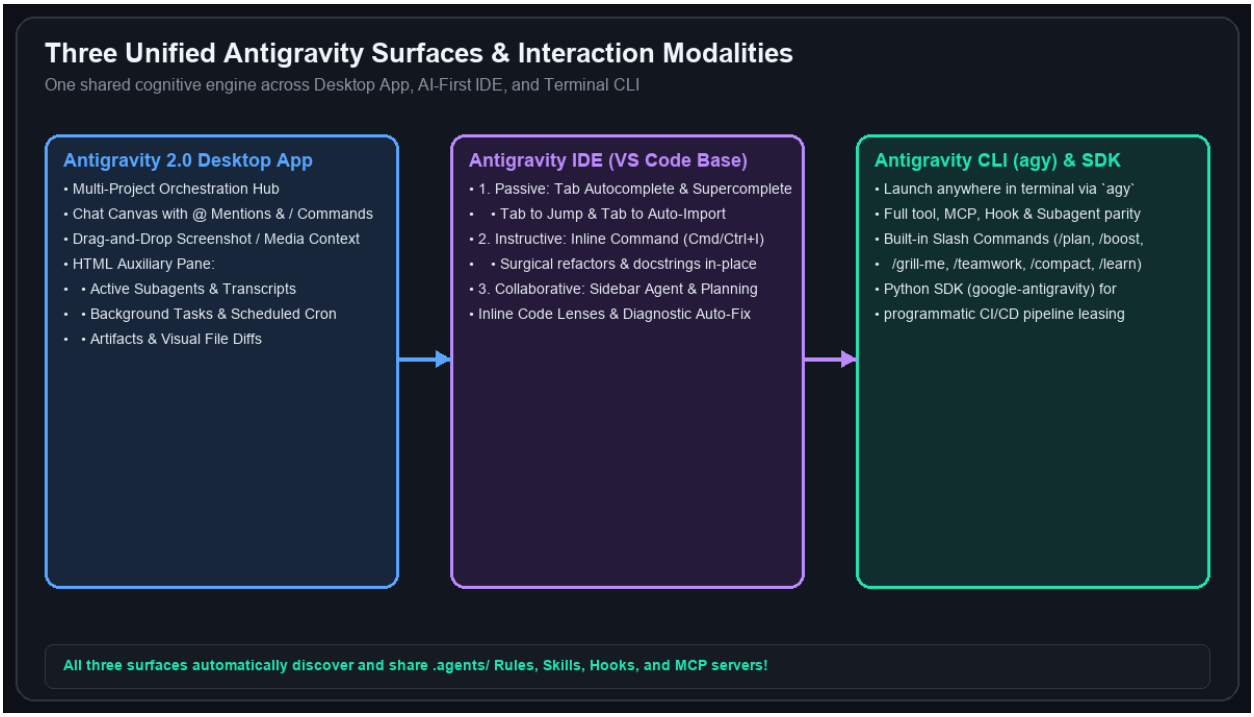
- **Hour 1 (00:00–00:58) — Foundations & Lab 01:** 6-minute Theory Capsule 1 (How Antigravity Works Internally + Surfaces & Modalities) followed by 52-minute **Lab 01: Root-Cause Triage & Financial Leakage Recovery**.
- **Hour 2 (00:58–01:58) — Domain Brain & Lab 02:** 6-minute Theory Capsule 2 (Hierarchical Rules & Progressive Disclosure Skills) followed by 54-minute **Lab 02: Encoding Industry Standards & Executive Risk Auditor Skill**.
- **Hour 3 (01:58–02:59) — Live Context, Hooks & Cloud Governance (Lab 03):** 6-minute Theory Capsule 3 (FastMCP, Deterministic PreToolUse Hooks & Live Gemini Enterprise / GCP Audit Demo) followed by 55-minute **Lab 03: Local FastMCP Server & Compliance Guardrails**.
- **Hour 4 (02:59–04:00) — Multi-Agent ADK, A2A, `adk web` & Terraform (Lab 04 Capstone):** 6-minute Theory Capsule 4 (Subagents, Google ADK, A2A Protocol, Local `adk web` Dev UI, and Terraform Cloud Run + Gemini Enterprise registration) followed by 55-minute **Lab 04 Capstone**.

Self-Healing Lab Checkpoints: A common challenge in hands-on workshops is when an attendee falls behind in Lab 1 and cannot start Lab 2. In both repositories (`antigravity-workshop-telecom` and `antigravity-workshop-retail`), every lab folder (`labs/lab-01-*` through `labs/lab-04-*`) includes an idempotent `./install.sh` (and `install.ps1` for Windows) script that automatically applies all prior labs' reference solutions and re-seeds the local SQLite database (`data/*.db`) in under 2 seconds.

2. Theory Deep Dive: How Google Antigravity Works Internally

When teaching developers, it is crucial to demystify what happens under the hood when they type a prompt in Google Antigravity. Antigravity is **not** a simple chat wrapper around an LLM—it is a deterministic, closed-loop agentic engineering runtime organized into five internal stages:

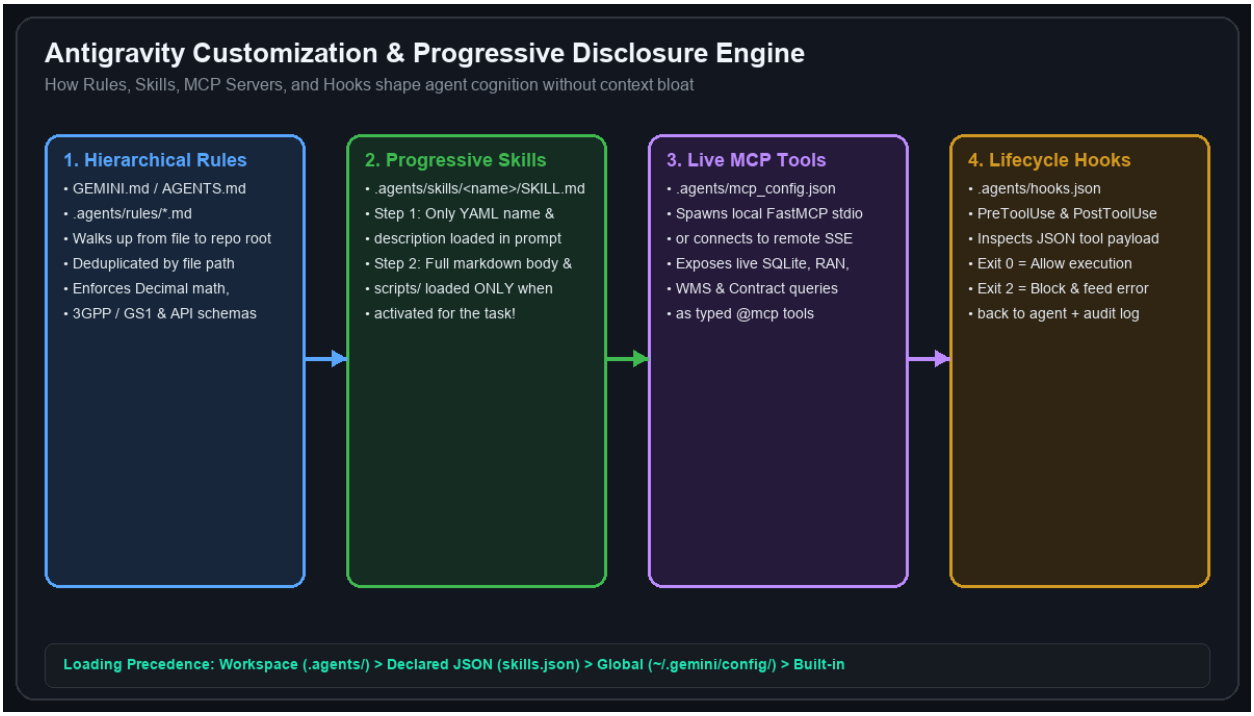
- **Stage 1 — Hierarchical Context Loader & Progressive Discovery:** Before sending a single token to the model, Antigravity walks up and down the workspace directory tree collecting `GEMINI.md` and `.agents/rules/*.md` files (deduplicating via canonical realpaths). Simultaneously, it scans `.agents/skills/*/SKILL.md` but extracts *only* the YAML frontmatter (`name` and `description` — ~50 tokens per skill). It also resolves explicit `@file`, `@folder`, and `@terminal` mentions and reads the conversation trajectory stored in `transcript.jsonl`.
- **Stage 2 — Gemini Enterprise Cognitive Planner:** The assembled context is sent over an encrypted enterprise channel to Gemini (governed by the user's Gemini Enterprise license). The planner uses **interleaved thinking** (`thinking` blocks) to decompose complex engineering problems, generate step-by-step architectural plans (`/plan`), and decide which tools (`read_file`, `edit_file`, `run_command`, MCP tools) or parallel subagents (`invoke_subagent`) to invoke.
- **Stage 3 — Deterministic Security Gate & Lifecycle Hooks:** Every tool call proposed by the model must pass through a deterministic local security gate *before* execution. Antigravity checks its Execution Policy (`strict`, `request-review`, or `proceed-in-sandbox`), validates path boundaries (blocking unapproved writes outside the workspace), and executes any registered `PreToolUse` shell hooks defined in `.agents/hooks.json`. It pipes the JSON tool payload to the hook's `stdin`: if the hook exits with code `0`, execution proceeds; if the hook exits with code `2`, Antigravity **hard-blocks** the action and feeds `stderr` back to the model!
- **Stage 4 — Sandboxed Tool Runtime & MCP Bridge:** Approved file edits are applied as exact character-sequence replacements. Shell commands (`pytest`, linters, CLI scripts) run inside an OS-level sandboxed process. External database or telemetry queries are dispatched over `stdio` or `SSE` using JSON-RPC 2.0 to configured Model Context Protocol (`MCP`) servers.
- **Stage 5 — Closed-Loop Verification & Self-Correction:** After a tool executes, `PostToolUse` hooks run and the stdout/stderr output (such as a failing `pytest` traceback) is fed back into the Cognitive Planner. Antigravity iterates automatically until all unit tests and verification suites pass.



2.1 Antigravity Surfaces & Developer Modalities

- **Passive Modality (IDE Tab / Supercomplete):** Context-aware multi-line code completion that predicts your next edit (`Tab`), jumps your cursor to the next logical edit location across files (`Tab-to-Jump`), and automatically adds missing Python imports (`Tab-to-Auto-Import`).
- **Instructive Modality (`Cmd+I` / `Ctrl+I` & Code Lenses):** Highlight a specific function (like the buggy CDR billing formula or ATP inventory formula), press `Cmd+I`, and give a targeted surgical instruction without leaving the editor. Above every function, clickable Code Lenses (`Explain`, `Modify`, `Write Tests`) accelerate routine workflows.
- **Collaborative & Autonomous Modality (Chat Canvas, Antigravity 2.0 Agent Manager & `agy` CLI):** Multi-file reasoning where developers use slash commands (`/plan`, `/grill-me`, `/boost`, `/teamwork`, `/learn`, `/compact`) and inspect the HTML Sidecar Panel (Subagents, Background Tasks, Artifacts, and File Changes).

3. Theory Deep Dive: Rules, Progressive Disclosure Skills, MCP, Hooks & Gemini Enterprise Governance

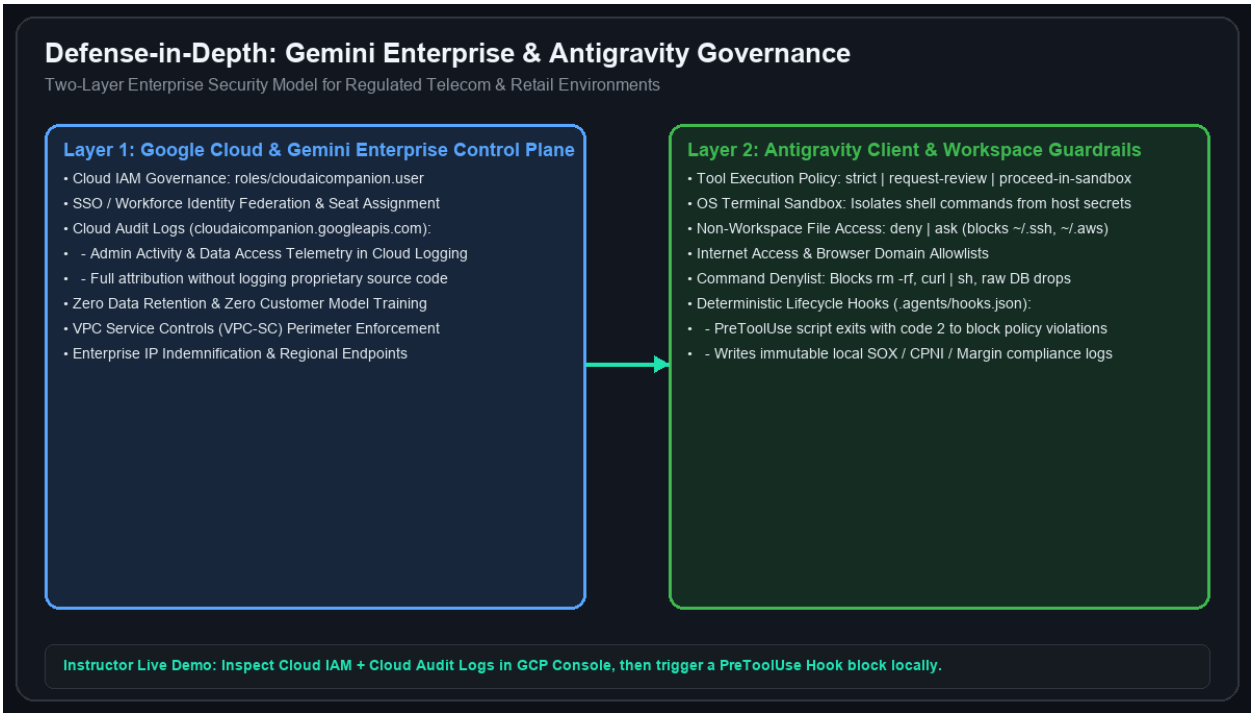


3.1 Why Progressive Disclosure Matters for Token Economics

If an enterprise team puts 40 pages of runbooks, SQL schemas, and audit scripts directly into `GEMINI.md`, every single prompt will consume tens of thousands of tokens, increasing latency and diluting model attention. Antigravity solves this by separating **Rules** from **Skills**:

- **Rules (`GEMINI.md` and `.agents/rules/*.md`):** Always-active architectural guardrails and non-negotiable domain invariants (e.g., 'Always use binary GiB = 2^30 for 3GPP CDR rating' or 'Never allow retail net price to drop below a 15% gross margin floor'). Keep these concise (< 200 lines).
- **Progressive Disclosure Skills (`.agents/skills/<skill-name>/SKILL.md`):** Task-specific workflows paired with executable helper scripts (`scripts/`). At startup, Antigravity reads *only* the `name` and `description` from the YAML frontmatter. When the user's request matches the skill's description, Antigravity dynamically reads the full `SKILL.md` instructions and executes its deterministic CLI scripts.

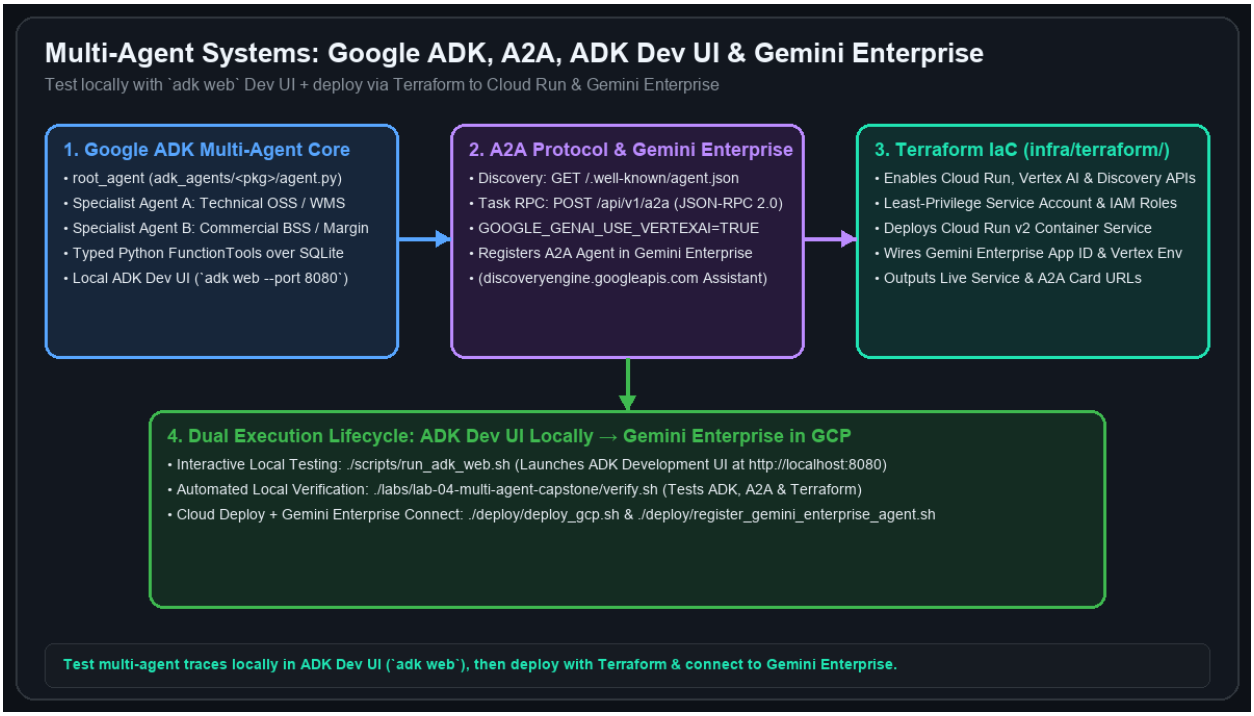
3.2 Live Enterprise Governance Demo: Gemini Enterprise + Google Cloud + Local Hooks



Attendees use Antigravity associated with their corporate **Gemini Enterprise** licenses, and the instructor has access to their Google Cloud environments. During Module 3, you will demonstrate how security and governance work across two complementary layers:

- **Layer 1 — Cloud Governance (Google Cloud Console Demo):** Open the GCP Console for the workshop project and show: (1) **Cloud IAM** (`roles/cloudaicompanion.user` and `roles/cloudaicompanion.admin`) binding developer identities to Gemini Enterprise seats; (2) **Cloud Audit Logs** filtering by `protoPayload.serviceName="cloudaicompanion.googleapis.com"`, proving that every API request is logged with principal email, timestamp, and caller IP; (3) **Data Privacy Guarantees**: under Gemini Enterprise, customer prompts and source code are **never** used to train foundation models and enforce zero data retention; (4) **VPC Service Controls (VPC-SC)** perimeters preventing data exfiltration.
- **Layer 2 — Client-Side Deterministic Hooks (`.agents/hooks.json`):** While Cloud IAM governs who can access the AI model, local `PreToolUse` hooks govern what the AI agent is allowed to do on the developer's machine or database. In Lab 03, attendees wire a Python hook (`scripts/audit_hook.py` in Telecom; `scripts/margin_guard_hook.py` in Retail) that inspects every tool call JSON payload on `stdin`, logs an immutable record to `logs/*.jsonl`, and exits with code `2` whenever a policy violation is attempted (e.g., an unapproved SLA credit > $5,000 in Telecom, or a price override below the 15% gross margin floor in Retail).

4. Theory Deep Dive: Multi-Agent Systems with Google ADK, A2A Protocol, `adk web` & Terraform



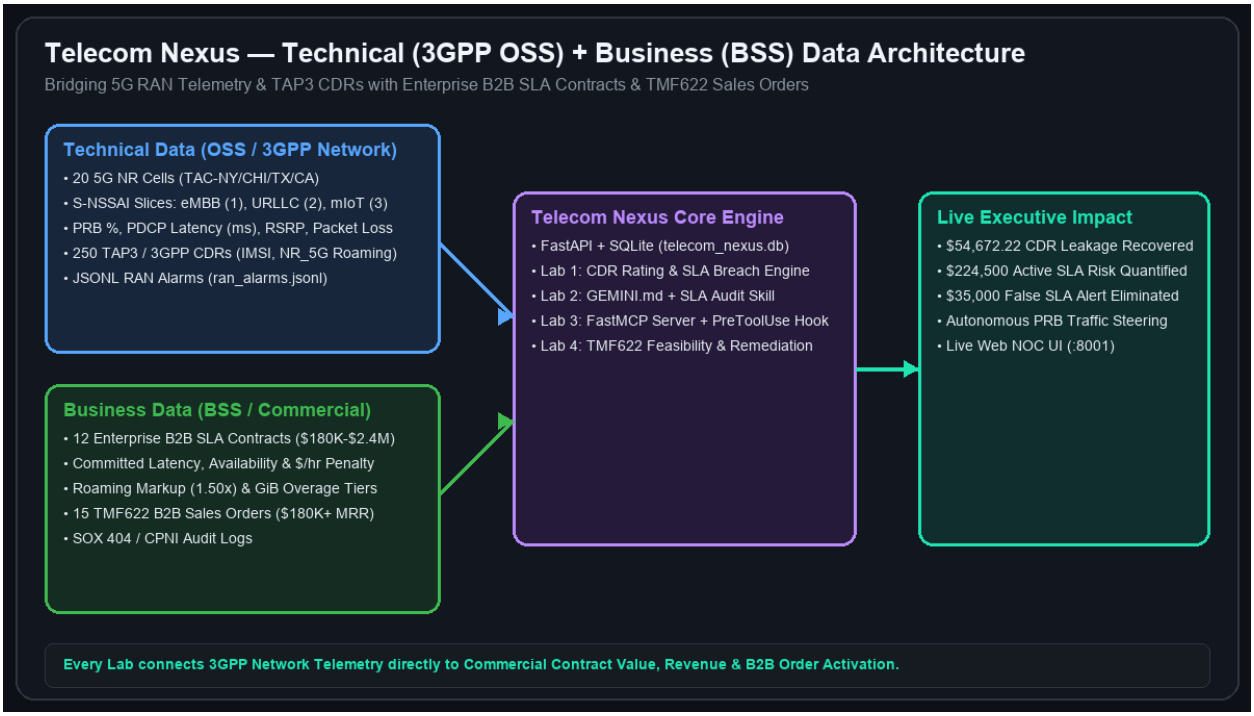
In Lab 04 (the Capstone Lab), attendees transition from using Antigravity as an engineering assistant to **designing and building their own production Multi-Agent System** using **Google ADK (`google-adk`)** and the **A2A (Agent2Agent) Protocol**, testing it locally with the **ADK Development UI (`adk web`)**, and preparing it for cloud deployment to **Google Cloud Run** and **Gemini Enterprise** using **Terraform**.

- **Google ADK (`google-adk`) Multi-Agent Hierarchy:** Google's Agent Development Kit provides composable primitives (`LlmAgent` / `Agent`) where a coordinator (`root_agent`) delegates specialized sub-tasks to domain sub-agents equipped with typed Python tool functions. In Telecom (`adk_agents/telecom_nexus/agent.py`), `root_agent` orchestrates `ran_telemetry_specialist` and `bss_sla_financial_specialist`. In Retail (`adk_agents/retail_pulse/agent.py`), `root_agent` orchestrates `inventory_atp_specialist` and `commercial_margin_otif_specialist`.
- **Local Interactive Testing with ADK Dev UI (`adk web`):** Both repositories include `./scripts/run_adk_web.sh` (which runs `adk web adk_agents --port 8080`). Opening `http://localhost:8080` launches the official ADK visual debugger where attendees can chat with `root_agent`, inspect live agent-to-subagent delegation traces, examine tool call inputs/outputs, and verify session state completely locally!
- **Deterministic Offline Fallback for 100% Local Reliability:** What if an attendee's laptop does not have an active Gemini API key configured inside their shell environment during `pytest`? Our orchestrator (`src/agents/adk_a2a_orchestrator.py`) builds the real `google.adk.agents.LlmAgent` hierarchy AND includes a deterministic execution pipeline so `pytest` and the local FastAPI endpoints work 100% offline with zero flaky tests, while `adk web` uses live Gemini models when `GOOGLE_API_KEY` or Vertex AI credentials are set.
- **A2A (Agent2Agent) Protocol Interoperability:** Why do we need A2A if we already have MCP? **MCP connects an Agent to Tools/Data** (stateless function calls), whereas **A2A connects an Agent to another Agent** (stateful task delegation, negotiation, and artifact generation). Every A2A-compliant agent publishes an **Agent Card** at `GET /.well-known/agent.json` declaring its skills, input/output modes, and authentication schemes, and accepts

JSON-RPC 2.0 `tasks/send` requests at `POST /api/v1/a2a`.

- **Cloud Deployment with Terraform & Gemini Enterprise Connection:** Every lab includes `infra/terraform/` (`main.tf`, `variables.tf`, `outputs.tf`) and `./deploy/deploy_gcp.sh`. Terraform enables required GCP APIs (`run.googleapis.com`, `artifactregistry.googleapis.com`, `cloudaicompanion.googleapis.com`, `discoveryengine.googleapis.com`), creates a least-privilege Service Account, and deploys the containerized FastAPI + ADK + A2A service to **Cloud Run v2**. Finally, `./deploy/register_gemini_enterprise_agent.sh` registers the Cloud Run A2A endpoint with **Gemini Enterprise** (`discoveryengine.googleapis.com`) so employees across the company can invoke the domain multi-agent system directly from their Gemini Enterprise workspace!

5. Track 1 Instructor Guide: Telecommunications Edition ('Telecom Nexus')



5.1 Understanding the Telecom Domain & Dataset (`data/telecom_nexus.db`)

If you do not come from a telecommunications background, here is the business story in plain English: Mobile operators make money not only from consumer cell phone plans, but increasingly from **Enterprise B2B 5G Network Slices**. A hospital chain, autonomous logistics port, or smart factory signs an Annual Contract (`ACV`, e.g., `$1,440,000/yr`) for a dedicated virtual slice of the 5G network (`S-NSSAI`), such as an Ultra-Reliable Low-Latency (`URLLC`, `SST=2`) slice. If a 5G cell tower (`gNodeB`) becomes congested (`PRB Utilization > 85%`) and air-interface delay (`PDCP latency`) exceeds the contract's threshold (e.g., `10 ms`), the operator breaches its **Service Level Agreement (SLA)** and owes thousands of dollars per hour in contractual penalties (`sla_penalty_per_hour_usd`). Meanwhile, the billing system (`BSS`) processes data usage records (`CDRs`) to charge customers for gigabytes consumed and roaming markups.

5.2 Telecom Lab-by-Lab Bug Explanations & Exact Solutions

Telecom Lab 01 — Root-Cause Triage & Revenue Leakage (`labs/lab-01-revenue-sla-triage`): • *Bug #1 (BSS CDR Rating Leakage in `src/services/cdr_rating.py`):* The starter code divides raw bytes by decimal `10**9` **(`1,000,000,000`) instead of binary `GiB` (`1024**3 = 1,073,741,824`), overcharging base volume by 7.37%, AND only applies `roaming_markup` to `'LTE'` while ignoring `'NR_5G'` visiting PLMN roaming usage (`visiting_plmn != home_plmn`). Fix: Divide `(uplink_bytes + downlink_bytes)` by `Decimal(1024 ** 3)` and apply `roaming_markup` to all visiting PLMN sessions including `'NR_5G'`. • *Bug #2 (RAN SLA Classifier in `src/services/sla_monitor.py`):* The starter SQL query forgets to filter `WHERE status = 'ACTIVE'` (including `'MAINTENANCE'` cells in breach reports) and uses `<` instead of `>` (`worst_latency < committed_latency`), inverting the breach condition! Fix: Filter `WHERE status = 'ACTIVE'` and trigger breach when `worst_latency > committed_latency` or `worst_jitter > committed_jitter`,

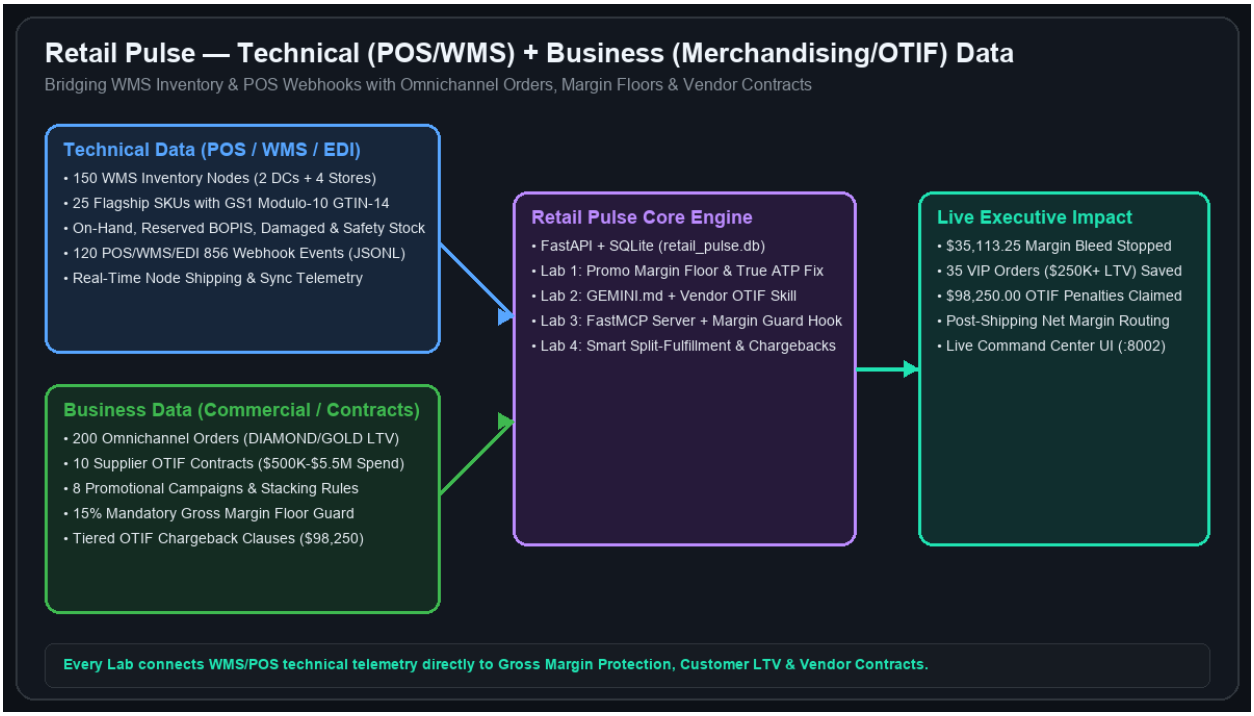
computing `hourly_penalty_exposure_usd`.

Telecom Lab 02 — Rules & Progressive Disclosure Skills (`labs/lab-02-rules-and-skills`): • Attendees create `GEMINI.md` and `.agents/rules/telecom-standards.md` encoding 3GPP S-NSSAI format, binary GiB billing (`1024**3 = 1,073,741,824`), `Decimal(ROUND_HALF_UP)`, and CPNI IMSI SHA-256 privacy hashing. • Attendees build `.agents/skills/sla-revenue-audit/SKILL.md` instructing Antigravity to execute `scripts/run_sla_audit.py` and synthesize an executive revenue risk briefing.

Telecom Lab 03 — FastMCP Server & Compliance Hooks (`labs/lab-03-mcp-and-hooks`): • Attendees complete `mcp_server/telecom_mcp.py` (`get_congested_slices`, `simulate_cdr_rerating`, etc.) and wire `.agents/mcp_config.json`. • Attendees configure `.agents/hooks.json` pointing `PreToolUse` to `scripts/audit_hook.py`. Any attempt to execute destructive SQL (`DROP`/`TRUNCATE`/`DELETE`), modify protected reference data (`data/`), or issue an unapproved SLA waiver exits with code `2` and logs to `logs/mcp_audit.jsonl`.

Telecom Lab 04 — Multi-Agent ADK, A2A, `adk web` & Terraform (`labs/lab-04-multi-agent-capstone`): • Attendees implement `src/services/order_feasibility.py` and `src/agents/adk_a2a_orchestrator.py` + `adk_agents/telecom_nexus/agent.py` (`root_agent` + `RANSliceCapacityAgent` + `BSSCommercialSLAAgent`), expose `GET /.well-known/agent.json` and `POST /api/v1/a2a` (`tasks/send`), test interactively with `./scripts/run_adk_web.sh`, run `pytest` (21/21 passing), and validate cloud deployment via `./deploy/deploy_gcp.sh --dry-run`.

6. Track 2 Instructor Guide: Omnichannel Retail Edition ('Retail Pulse')



6.1 Understanding the Retail Domain & Dataset (`data/retail_pulse.db`)

If you do not come from a retail or supply chain background, here is the business story in plain English: Modern omnichannel retailers sell products (`SKUs` identified by 14-digit `GS1 GTIN-14` barcodes) across physical stores (`POS`) and online e-commerce (`BOPIS` — Buy Online, Pick Up In Store, and home delivery from Distribution Centers `DCs`). Two catastrophic bugs commonly destroy retail profitability: (1) **Phantom Inventory & VIP Stockouts**, where the Order Management System (`OMS`) looks only at `on_hand - reserved` and forgets to subtract `safety_stock` and `unprocessed_webhook_delta_qty`—causing the retailer to oversell items that aren't actually available to high-value `VIP_PLATINUM` / `VIP_GOLD` customers (`Customer LTV`); and (2) **Promo Stacking Margin Collapse**, where combining multiple discount codes additively instead of multiplicatively drives the net selling price below the SKU's contractual **Minimum Margin Floor (`contract_min_margin_floor_pct`, e.g. 15%–28%)**. Meanwhile, when suppliers deliver shipments late or short (`EDI 856 ASN` with `OTIF`), the retailer must re-route at-risk VIP orders to optimal fulfillment nodes.

6.2 Retail Lab-by-Lab Bug Explanations & Exact Solutions

Retail Lab 01 — ATP Inventory & Promo Margin Triage (`labs/lab-01-margin-inventory-triage`): • *Bug #1 (Phantom ATP Inventory in `src/services/inventory_atp.py`):* Starter code computes `on_hand_qty - reserved_qty`, ignoring `safety_stock_qty` and `unprocessed_webhook_delta_qty`, causing overselling on BOPIS orders. Fix: Compute `raw_atp = on_hand - reserved - safety_stock - webhook_delta` (`available_to_promise_qty = max(0, raw_atp)`). • *Bug #2 (Additive Promo Stacking & Margin Violation in `src/services/promo_pricing.py`):* Starter code uses `float`, sums promo percentages additively, allows stacking non-stackable promos, and ignores `contract_min_margin_floor_pct`. Fix: Implement GS1 `GTIN-14` Modulo-10 check-digit validation

(`validate_gs1_gtin14`), use `Decimal` (`ROUND_HALF_UP`), select the single best non-stackable promo or compound stackable promos multiplicatively (`price * (1 - d1) * (1 - d2)`), and clamp to `min_allowed_price = unit_cost / (1 - floor_pct)`.

Retail Lab 02 — Rules & Progressive Disclosure Skills (`labs/lab-02-rules-and-skills`): • Attendees create `GEMINI.md` and `.agents/rules/retail-standards.md` codifying GS1 GTIN-14 Modulo-10 math, strict ATP deductions (`safety_stock` + `webhook_delta`), multiplicative promo compounding, the margin floor, and Customer PII masking. • Attendees build `.agents/skills/margin-otif-audit/SKILL.md` instructing Antigravity to run `scripts/run_otif_margin_audit.py` and produce an executive margin & stockout audit.

Retail Lab 03 — FastMCP Server & Margin Guardrail Hooks (`labs/lab-03-mcp-and-hooks`): • Attendees complete `mcp_server/retail_mcp.py` (`simulate_promo_margin_guard`, `list_at_risk_vip_orders`, etc.) and configure `.agents/mcp_config.json`. • Attendees wire `.agents/hooks.json` (`PreToolUse` -> `scripts/margin_guard_hook.py`), which blocks any price override below the contractual gross margin floor or destructive SQL command with exit code `2` and writes to `logs/margin_guard_audit.jsonl`.

Retail Lab 04 — Multi-Agent ADK, A2A, `adk web` & Terraform (`labs/lab-04-multi-agent-capstone`): • Attendees implement `src/services/fulfillment_router.py` (`evaluate_candidates`, `optimize_at_risk_orders`) and `src/agents/adk_a2a_orchestrator.py` + `adk_agents/retail_pulse/agent.py` (`root_agent` + `WMSInventoryATPAgent` + `MerchandisingMarginOTIFAgent`), expose `GET /.well-known/agent.json` and `POST /api/v1/a2a` (`tasks/send`), test interactively in `./scripts/run_adk_web.sh`, run `pytest` (20/20 passing), and validate Terraform deployment via `./deploy/deploy_gcp.sh --dry-run`.

7. Master Acronym & Terminology Glossary (Labs, Architecture & Industry Datasets)

Use this reference section whenever an attendee asks what a technical, cloud, telecommunications, or retail acronym means in the slides, source code, or SQLite/JSONL datasets.

7.1 AI, Antigravity, Multi-Agent & Google Cloud Acronyms

Acronym	Full Term	Plain-English Explanation & Workshop Role
AGY	Google Antigravity	Google's agentic AI software engineering platform combining an IDE, Autonomous Agent Manager (Antigravity 2.0), and CLI (`agy`).
ADK	Agent Development Kit	Google's open-source Python framework (`google-adk`) for building, orchestrating, and testing hierarchical multi-agent systems (`LlmAgent`, `SequentialAgent`, `adk web`).
A2A	Agent2Agent Protocol	Open interoperability standard allowing independent AI agents to discover capabilities (`GET /.well-known/agent.json`) and delegate tasks via JSON-RPC 2.0 (`tasks/send`).
MCP	Model Context Protocol	Open standard connecting AI agents to external databases, APIs, and tools over `stdio` or `SSE` without hardcoding integrations.
FastMCP	FastMCP Python SDK	High-level Python library (`mcp.server.fastmcp`) using `@mcp.tool()` decorators to expose typed Python functions as MCP tools.
JSON-RPC	JSON Remote Procedure Call 2.0	Stateless, light-weight remote procedure call protocol encoded in JSON; used as the transport payload for both MCP and A2A.
SSE	Server-Sent Events	HTTP streaming standard used by remote MCP and A2A servers to push real-time progress and token updates to clients.
STDIO	Standard Input / Output	Local inter-process communication stream used by Antigravity to spawn and communicate with local MCP servers and lifecycle hooks.
AST	Abstract Syntax Tree	Tree representation of source code structure used by Antigravity's Supercomplete and refactoring engine to make syntax-aware edits.
IAM	Identity and Access Management	Google Cloud security framework controlling who (users/service accounts) has what permissions (roles like `roles/cloudaicompanion.user`) on which resources.
VPC-SC	VPC Service Controls	Google Cloud security perimeter technology that prevents data exfiltration from authorized cloud resources and Gemini Enterprise APIs.
IaC	Infrastructure as Code	Practice of provisioning cloud infrastructure (Cloud Run, IAM, Artifact Registry) via declarative configuration files using Terraform (`.tf`).
HCL	HashiCorp Configuration Language	Declarative language used in Terraform files (`main.tf`, `variables.tf`, `outputs.tf`) to define cloud resources.
LLM	Large Language Model	Foundation AI model (such as Gemini 2.5 Pro / Flash) that powers Antigravity's reasoning, planning, and tool calling.
PII	Personally Identifiable Information	Sensitive subscriber or customer personal data (names, phone numbers, emails) that must be masked or hashed for privacy compliance.

7.2 Telecommunications Track Acronyms (3GPP 5G RAN, GSMA Billing, TM Forum & Commercial SLA)

Acronym	Full Term	Plain-English Explanation & Dataset Context
3GPP	3rd Generation Partnership Project	Global standards body that defines 5G New Radio (NR), core network, slicing, and telecom charging specifications.
5G NR	5G New Radio	Global wireless air interface standard developed by 3GPP for 5G mobile networks.
RAN	Radio Access Network	The wireless network layer (antennas, radios, gNodeBs) connecting user devices (phones, IoT sensors) to the core network.
gNodeB	Next Generation Node B	5G wireless base station tower/node (e.g., `GNB-DAL-101` in our dataset) that transmits and receives 5G radio signals across cell sectors.
TAC	Tracking Area Code	3GPP geographical grouping of cell sites used to track device location and paging (e.g., `41010` in the Telecom Nexus dataset).
PLMN	Public Land Mobile Network	Unique identifier for a mobile operator's network, formed by concatenating `MCC` + `MNC` (e.g., `310-410`).
MCC	Mobile Country Code	3-digit ITU code identifying the country of a mobile network subscriber (e.g., `310` = United States, `748` = Uruguay).
MNC	Mobile Network Code	2- or 3-digit code identifying the specific mobile carrier within a country (combined with MCC to form PLMN).
S-NSSAI	Single Network Slice Selection Assistance Information	3GPP 5G identifier (`SST` + `SD`, e.g., `1-010203` or `2-ABCDEF`) that uniquely identifies a virtual 5G network slice.
SST	Slice/Service Type	8-bit integer inside S-NSSAI defining standard slice behavior: `1` = eMBB (broadband), `2` = URLLC (low latency), `3` = mIoT (IoT).
SD	Slice Differentiator	24-bit hex value (6 hex characters, e.g., `010203`) inside S-NSSAI that distinguishes multiple slices of the same SST for specific enterprise clients.
eMBB	Enhanced Mobile Broadband	5G slice type (`SST=1`) optimized for high-bandwidth data speeds (video streaming, enterprise branch connectivity).
URLLC	Ultra-Reliable Low-Latency Communications	5G slice type (`SST=2`) requiring sub-10ms PDCP latency and 99.999% reliability (autonomous robotics, smart grids, telesurgery).
mIoT	Massive Internet of Things	5G slice type (`SST=3`, also called mMTC) optimized for high connection density of low-power sensors and smart meters.
PRB	Physical Resource Block	Smallest unit of radio frequency/time spectrum allocated by a 5G gNodeB scheduler. `PRB Utilization %` > 85% indicates cell congestion.
PDCP	Packet Data Convergence Protocol	5G radio protocol layer handling header compression, encryption, and packet delivery; `pdcp_latency_ms` measures air-interface delay.
RSRP	Reference Signal Received Power	Measure of 5G signal strength received by a device, expressed in negative decibel-milliwatts (`dBm`, e.g., `-85 dBm` is good, `-115 dBm` is weak).

Acronym	Full Term	Plain-English Explanation & Dataset Context
SINR	Signal-to-Interference-plus-Noise Ratio	Measure of 5G radio signal quality in decibels (`dB`). High PRB congestion lowers SINR and causes packet drops.
5QI	5G QoS Identifier	Scalar integer (e.g., `5QI=1` for voice, `5QI=82` for URLLC automation) defining packet forwarding latency and loss budgets in 5G.
QoS	Quality of Service	Traffic prioritization mechanisms ensuring mission-critical enterprise slices receive guaranteed bandwidth and low latency.
RRC	Radio Resource Control	3GPP control-plane protocol managing connection establishment, handovers, and active connected device counts (`rrc_connected_ues`).
OSS	Operations Support Systems	Telecom software systems managing network fault alarms, RAN performance telemetry, and cell provisioning.
BSS	Business Support Systems	Telecom commercial systems handling customer orders, billing, CDR rating, B2B contracts, and revenue assurance.
CDR	Call Detail Record / Charging Data Record	Usage record generated by the network for every voice/data session, containing `IMSI`, `MSISDN`, bytes transferred (`bytes_uplink`, `bytes_downlink`), and cell ID.
TAP3	Transferred Account Procedure v3	GSMA standard file format used by mobile operators worldwide to exchange and bill international roaming CDRs.
IMSI	International Mobile Subscriber Identity	Unique 15-digit number stored on a SIM/eSIM card (`MCC`+`MNC`+`MSIN`) identifying a mobile subscriber on the network.
MSISDN	Mobile Station International Subscriber Directory Number	The dialable E.164 telephone number associated with a subscriber SIM (e.g., `+14155550199`). Regulated as CPNI.
IMEI	International Mobile Equipment Identity	Unique 15-digit serial number identifying the physical mobile handset or 5G IoT modem.
APN / DNN	Access Point Name / Data Network Name	Logical network gateway name in 4G (`APN`) and 5G (`DNN`) that routes device traffic to the internet or a private enterprise slice.
GiB vs GB	Gibibyte vs Gigabyte	`1 GiB = 1,073,741,824 bytes` (2^{30}, binary standard used in 3GPP CDR rating) vs `1 GB = 1,000,000,000 bytes` (10^9). Confusing them causes a 7.37% billing error!
SLA	Service Level Agreement	Contractual uptime and latency guarantee with an enterprise B2B customer, specifying hourly financial penalty rates (`sla_penalty_per_hour_usd`) on breach.
ACV / MRR	Annual Contract Value / Monthly Recurring Revenue	Commercial metrics measuring the total yearly (`ACV`) and monthly (`MRR`) dollar value of an enterprise telecom contract.
TMF622	TM Forum Product Ordering API	Telecom industry standard REST API specification for B2B product and network slice sales orders.
TMF621	TM Forum Trouble Ticket API	Telecom industry standard REST API specification for creating and tracking network incident and SLA breach tickets.
CPNI	Customer Proprietary Network Information	Legally protected telecom data (call logs, location, MSISDN, IMSI) requiring strict privacy masking and SHA-256 hashing.

Acronym	Full Term	Plain-English Explanation & Dataset Context
SOX	Sarbanes-Oxley Act	Financial compliance regulation requiring strict audit trails and executive approval before issuing large SLA billing credits (e.g., > $5,000).

7.3 Retail Track Acronyms (GS1 Barcodes, Omnichannel WMS/POS, Supply Chain & Commercial)

Acronym	Full Term	Plain-English Explanation & Dataset Context
POS	Point of Sale	In-store checkout and payment terminal system generating real-time sales transactions and store inventory decrements.
WMS	Warehouse Management System	Software controlling inventory storage, putaway,damaged-hold quarantine, picking, and shipping inside Distribution Centers and stores.
OMS	Order Management System	Central brain routing e-commerce and BOPIS customer orders to the optimal fulfillment node based on real-time ATP inventory.
ATP	Available-to-Promise	Net sellable inventory quantity that can safely be promised to a customer: `ATP = max(0, on_hand - reserved_bopis - damaged_hold - safety_stock)`.
BOPIS	Buy Online, Pick Up In Store	Omnichannel fulfillment model where customers order online and collect items at a local retail store; requires strict inventory reservation (`reserved_bopis_qty`).
DC / FC	Distribution Center / Fulfillment Center	Regional warehouse node (e.g., `DC-EAST-01`) that replenishes retail stores and fulfills direct-to-consumer online orders.
GS1	Global Standards 1	International non-profit organization maintaining global standards for barcodes (`GTIN`), `EPC/RFID`, and supply chain `EDI` messaging.
GTIN-14	Global Trade Item Number (14-digit)	14-digit GS1 product identifier encoded in barcodes and RFID tags; the 14th digit is a mathematically calculated Modulo-10 check digit.
SKU	Stock Keeping Unit	Internal retailer alphanumeric code identifying a specific product style, size, and color (e.g., `SKU-LUX-BAG-001`).
UPC / EAN	Universal Product Code / European Article Number	12-digit (`UPC-A`) or 13-digit (`EAN-13`) consumer package barcodes that are zero-padded to 14 digits to form a canonical `GTIN-14`.
RFID / EPC	Radio-Frequency Identification / Electronic Product Code	Wireless item-level inventory tag transmitting serialized product and GTIN data in stores and warehouses.
EDI 856	Electronic Data Interchange 856 (ASN)	Standard automated electronic message sent by a supplier when a shipment leaves their dock, detailing expected cartons, SKUs, and arrival time.
ASN	Advance Ship Notice	The human-readable supply chain document represented electronically by an `EDI 856` message.
OTIF	On-Time In-Full	Core supplier performance KPI measuring the percentage of purchase orders delivered both on schedule (`On-Time`) and with 100% of ordered units (`In-Full`).
LTV	Customer Lifetime Value	Total projected net profit attributed to the entire future relationship with a customer (e.g., `$18,400` for a `DIAMOND` VIP shopper).

Acronym	Full Term	Plain-English Explanation & Dataset Context
VIP Tier	Loyalty Tier (DIAMOND, GOLD, SILVER, BRONZE)	Customer segmentation classification used to prioritize scarce ATP inventory allocation and SLA expedite remedies during stockouts.
COGS	Cost of Goods Sold	Direct wholesale unit cost paid by the retailer to acquire or manufacture a product (`unit_cost_usd`).
Gross Margin %	Gross Profit Margin Percentage	Profitability ratio calculated as `(net_price - unit_cost_usd) / net_price`. In Retail Pulse, corporate policy enforces a strict `>= 15.0%` floor.
Promo Stacking	Promotional Discount Stacking	Applying multiple promo codes to a single order; must distinguish `EXCLUSIVE` promos from `STACKABLE` promos and apply sequentially (compound), never additively.

8. Live Troubleshooting Playbook: 16 Questions Attendees Ask When Things Go Wrong

This section prepares the instructor for every realistic failure mode during Labs 01–04 (environment setup, port conflicts, Gemini Enterprise IAM, floating-point math bugs, progressive disclosure skill discovery, MCP stdio corruption, hook exit code 2 vs 1, `adk web` credentials, and Terraform dry-run).

[Environment & Setup (All Labs)] Q1: 'When I run ./labs/lab-01-.../install.sh, I get python3: command not found, No module named venv, or an old Python version error.'

Root Cause: Attendee laptop is missing Python 3.10+ on PATH or lacks python3-venv on Ubuntu/Debian/WSL.

Instructor Answer & Exact Fix:
1. Check if `uv` is installed (`uv --version`). If so, run `uv venv .venv --python 3.11 && source .venv/bin/activate && uv pip install -r requirements.txt`.
2. On macOS with Homebrew: `brew install python@3.11`.
3. On Windows: Run PowerShell script `.\labs\lab-01-...\install.ps1` instead of `.sh`.
4. Always ensure commands are run using `.venv/bin/python3` and `.venv/bin/pytest`.

[Environment & Setup (All Labs)] Q2: 'When I start the FastAPI dashboard (uvicorn src.main:app --port 8000), it crashes with [Errno 48] Address already in use (macOS/Linux) or WinError 10048 (Windows).'

Root Cause: Another process or a previous terminal tab is already listening on port 8000 (or port 8080 for `adk web`).

Instructor Answer & Exact Fix:
• On macOS/Linux, free the port immediately: `lsof -ti:8000 | xargs kill -9` (or for port 8080: `lsof -ti:8080 | xargs kill -9`).
• Alternatively, run on a different port: `.venv/bin/uvicorn src.main:app --reload --port 8005`.

[Environment & Setup (All Labs)] Q3: 'I accidentally corrupted my SQLite database (data/*.db) or deleted part of a service file while experimenting. How do I recover quickly?'

Root Cause: Attendee ran a destructive query or bad edit during exploration.

Instructor Answer & Exact Fix:
• Re-run the current lab's installer: `./labs/lab-0X-.../install.sh`.
• Explain that every `install.sh` script is **self-healing and idempotent**: it automatically restores the clean starter state for the current lab, applies verified solutions for all prior labs, and re-seeds the SQLite database (`python3 data/seed_telecom_data.py` or `python3 data/seed_retail_data.py`) in 2 seconds!

[Environment & Setup (All Labs)] Q4: 'I didn't finish Lab 1 in time and now the class is moving to Lab 2. Am I stuck?'

Root Cause: Normal pacing variance across attendees in a hands-on coding workshop.

Instructor Answer & Exact Fix:
• Reassure the attendee: 'Not at all! Just run `./labs/lab-02-rules-and-skills/install.sh`.'
• Every lab's `install.sh` automatically copies the reference solution from prior labs into `src/` before setting up the new lab. And if anyone wants to see the full solution to the current lab, they can run `./labs/lab-0X-.../apply_solution.sh`.

[Antigravity & Gemini Enterprise Auth] Q5: 'Antigravity gives a 403 PermissionDenied error or says cloudaicompanion.googleapis.com is not enabled for my account.'

Root Cause: The attendee's corporate Google identity is either logged into a personal Gmail profile or missing the IAM role binding on the workshop GCP project.

Instructor Answer & Exact Fix:
1. Verify the attendee is signed into Antigravity using their corporate email associated with the workshop.
2. In the Instructor GCP Console -> **IAM & Admin**, grant the attendee's email two roles: `Gemini for Google Cloud User` (`roles/cloudaicompanion.user`) and `Service Usage Consumer` (`roles/serviceusage.serviceUsageConsumer`).
3. Ensure `cloudaicompanion.googleapis.com` is enabled in **APIs & Services**.

[Lab 01 — Telecom Math & Logic] Q6 (Telecom): 'Antigravity updated cdr_rating.py, but pytest still fails on the CDR charge amount by ~7.37% or on NR_5G roaming!'

Root Cause: Two classic Telecom BSS pitfalls in `src/services/cdr_rating.py`: (1) Dividing total bytes by decimal GB (`10**9 = 1,000,000,000`) instead of 3GPP binary GiB (`2**30 = 1024**3 = 1,073,741,824`); or (2) Only applying `roaming_markup` when `rat_type == 'LTE'` and ignoring `'NR_5G'` visiting PLMN roaming.

Instructor Answer & Exact Fix:
• Show the exact formula using `Decimal`: `volume_gib = Decimal(uplink_bytes + downlink_bytes) / Decimal(1024 ** 3)`.
• Apply `roaming_markup` whenever `visiting_plmn != home_plmn` across both `'LTE'` and `'NR_5G'`, and quantize to `Decimal('0.01')` using `ROUND_HALF_UP` only at the end!

[Lab 01 — Telecom Math & Logic] Q7 (Telecom): 'Why is sla_monitor.py either including MAINTENANCE cells or missing active URLLC latency breaches?'

Root Cause: In `src/services/sla_monitor.py`, the starter query omits `WHERE status = 'ACTIVE'` and uses `<` instead of `>` (`worst_latency < committed_latency`).

Instructor Answer & Exact Fix:
• Ensure the SQL query filters `WHERE c.status = 'ACTIVE'` so maintenance windows are excluded.
• Trigger a breach when `worst_latency > committed_latency` or `worst_jitter > committed_jitter`, and compute `hourly_penalty_exposure_usd`.

[Lab 01 — Retail Math & Logic] Q8 (Retail): 'In promo_pricing.py, I enforced a margin floor using unit_cost * (1 + floor_pct), so why is pytest saying my minimum price is wrong?'

Root Cause: Classic confusion between **Markup** (`cost * (1 + pct)`) and **Gross Margin** (`(price - cost) / price = pct`).

Instructor Answer & Exact Fix:
• Write both formulas on the screen—attendees love this clarification:
- **Markup** divides profit by cost: `price = cost * 1.15` (if cost is $85, 15% markup gives $97.75, which is only a 13.04% gross margin!).
- **Gross Margin** divides profit by net selling price: `(price - cost) / price >= floor_pct`, which algebraically rearranges to `min_price = unit_cost / (Decimal('1') - floor_pct)` (if cost is $85 and floor is 15%, `$85 / 0.85 = $100.00`).
• Also confirm `stackable` promos are applied multiplicatively (`price = price * (1 - d1) * (1 - d2)`), never added (`d1 + d2`).

[Lab 01 — Retail Math & Logic] Q9 (Retail): 'Why does my GS1 GTIN-14 barcode validator reject valid 14-digit barcodes in inventory_atp.py?'

Root Cause: Reversing the 3-and-1 multiplier weights in the GS1 Modulo-10 algorithm.

Instructor Answer & Exact Fix:
• For a 14-digit `GTIN-14`, the first 13 digits (`digits[:13]`) are the payload and the 14th digit (`digits[13]`) is the check digit.
• Moving from left to right across the first 13 digits (index `0` to `12`), even indices (`0, 2, 4, ... 12`) are multiplied by **`3`**, and odd indices (`1, 3, 5, ... 11`) are multiplied by **`1`**. Then `check_digit = (10 - (total % 10)) % 10`.

[Lab 02 — Rules & Skills] Q10: 'I created SKILL.md inside .agents/skills/, why doesn't Antigravity discover or activate my skill when I prompt it?'

Root Cause: Progressive disclosure discovery relies strictly on the YAML frontmatter at the top of `SKILL.md` and the exact directory path.

Instructor Answer & Exact Fix:
Check these 3 items with the attendee:
1. **Exact Path & Case:** Must be `.agents/skills/<skill-name>/SKILL.md` (`SKILL.md` in ALL CAPS, inside `.agents/skills/`).
2. **Valid YAML Frontmatter:** Line 1 of `SKILL.md` must be `---`, followed by `name:` and `description:`, closed by `---`.
3. **Trigger Keywords in Description:** Antigravity only loads `name` and `description` initially. Make sure `description` clearly states when to activate (e.g., *'Use when generating an executive SLA revenue risk audit...'*).

[Lab 02 — Rules & Skills] Q11: 'When Antigravity runs my skill script (.agents/skills/.../scripts/run_audit.py), it fails with ModuleNotFoundError: No module named src.'

Root Cause: Python does not automatically add the workspace root to `sys.path` when executing a script nested four directories deep inside `.agents/skills/<name>/scripts/`.

Instructor Answer & Exact Fix:
• Either run the script with `PYTHONPATH=. .venv/bin/python3 .agents/skills/.../scripts/run_audit.py`, OR
• Add `sys.path.insert(0, str(Path(__file__).resolve().parents[4]))` at the top of `run_audit.py` before importing from `src.*`.

[Lab 03 — FastMCP & Hooks] Q12: 'My FastMCP server works when I import it in Python, but Antigravity says MCP connection failed or invalid JSON-RPC frame!'

Root Cause: The #1 cause of `stdio` MCP bugs: someone added a `print('Connected to DB...')` statement that writes to `stdout`!

Instructor Answer & Exact Fix:
• Explain how `stdio` MCP transport works: Antigravity spawns `mcp_server/*_mcp.py` as a subprocess and communicates over `stdin` and `stdout` using strict JSON-RPC 2.0 messages.
• If any code prints plain text to `stdout`, it corrupts the JSON-RPC stream! Any debug logs inside an MCP server MUST use `print(..., file=sys.stderr)` or Python's `logging` module configured to `stderr`.
• Also verify `.agents/mcp_config.json` uses `.venv/bin/python3` (not system `python3` which might lack the `mcp` package).

[Lab 03 — FastMCP & Hooks] Q13: 'My PreToolUse hook script exits with code 1 on a violation, why didn't Antigravity treat it as a deterministic policy block?'

Root Cause: In Antigravity's hook specification, exit code `1` indicates a non-fatal script crash/warning, whereas **exit code `2`** is the explicit signal for a deterministic policy block!

Instructor Answer & Exact Fix:
• Review the Antigravity Hook Exit Code Contract:
- `sys.exit(0)` -> Action is compliant; proceed with tool execution.
- `sys.exit(2)` -> **Deterministic Policy Block!** Antigravity aborts the tool call immediately and feeds the hook's `stderr` explanation back to the agent.
• Also ensure the script reads the tool payload via `payload = json.load(sys.stdin)` and writes an audit entry to `logs/*.jsonl` before calling `sys.exit(2)`.

[Lab 04 — ADK, adk web, A2A & Terraform] Q14: 'When I launch ./scripts/run_adk_web.sh (http://localhost:8080) and send a chat message in the ADK Dev UI, it asks for GOOGLE_API_KEY or Vertex AI credentials. How does that work?'

Root Cause: `pytest` and the FastAPI A2A endpoint include a deterministic offline engine so local tests always pass without external API keys, whereas `adk web` invokes the live Gemini LLM inside `google.adk.agents.LlmAgent`.

Instructor Answer & Exact Fix:
• To chat live with Gemini inside `adk web`, configure either:
- **Option A (Vertex AI / Gemini Enterprise GCP Project):**
`export GOOGLE_GENAI_USE_VERTEXAI=TRUE`
`export GOOGLE_CLOUD_PROJECT=<your-workshop-gcp-project-id>`
`export GOOGLE_CLOUD_LOCATION=us-central1`
`gcloud auth application-default login`
- **Option B (Gemini API Key):**
`export GOOGLE_API_KEY=<your-api-key>`
• Then restart `./scripts/run_adk_web.sh` and open `http://localhost:8080`!

[Lab 04 — ADK, adk web, A2A & Terraform] Q15: 'In the ADK Dev UI (http://localhost:8080), the agent dropdown menu is empty or doesn't list telecom_nexus / retail_pulse!'

Root Cause: `adk web` expects a specific directory convention (`adk_agents/<agent_name>/__init__.py` and `agent.py` exporting a variable named `root_agent`).

Instructor Answer & Exact Fix:
• Always launch `adk web` via `./scripts/run_adk_web.sh` (which runs `.venv/bin/adk web adk_agents --port 8080` from the repository root).
• Verify that `adk_agents/<agent_name>/agent.py` defines `root_agent = LlmAgent(...)` and `adk_agents/<agent_name>/__init__.py` contains `from . import agent`.

[Lab 04 — ADK, adk web, A2A & Terraform] Q16: 'When I run ./deploy/deploy_gcp.sh to test the Terraform deployment, I don't have Docker Desktop running or GCP project owner permissions on my laptop.'

Root Cause: Attendee ran `./deploy/deploy_gcp.sh` without `--dry-run` on a laptop without Docker or active `gcloud` credentials.

Instructor Answer & Exact Fix:
• For local verification on attendee laptops, run `./deploy/deploy_gcp.sh --dry-run` (or `./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run`).
• Explain that `--dry-run` validates all Terraform HCL files (`infra/terraform/main.tf`, `variables.tf`, `outputs.tf`), Dockerfile, and Gemini Enterprise registration payloads 100% locally without requiring Docker daemon or live GCP privileges!